

كسر حماية ميزان Pro بـ x32dbg

من الصفر إلى المية — دليل الفريق والمشرفين على الورشة

الفريق	علي العطيفي · ماجد الدميني · المعتصم الراعي — دفعة 2026
المادة	الهندسة العكسية — المشروع العملي (فصل 4)
المشرف	م. عبدالكريم الباشيري
الهدف	ميزان crackme — Pro 3.0.0.0 من بنائنا، win-x86 / .NET 6
الحجم	عرض 35 دقيقة + تطبيق مباشر + إشراف على 3 مجموعات

حدود ما سنفعله · خطأ شائع — تجنبه

كل ما في هذا الدليل يطبق على برنامج كتبناه نحن خصيصاً لهذا الغرض (crackme). (أكاديمي تطبيق أي خطوة هنا على برنامج لا تملكه = مخالفة نظامية وأكاديمية؛ لا تفعلها، ولا تنشر أي أداة ناتجة عنها.
سنظهر الثغرات ونوقش الدفاع ضدها — هذه هي المادة، لا « كيف نسرق برنامجاً ».

كيف تقرأ هذا الدليل

ثلاث مسارات حسب الوقت المتبقي لك

الدليل مبني ليقراً من أوله لآخره في ساعة، لكن **غداً عندك .عرض** اختر مسارك :الآن

المسار	اقرأ الوقت	لمن
أ — النجاة السريعة	30 د	يجرب لم من إطلاقاً x32dbg
ب — الفهم الكامل	90 د	يشرح أن يريد من فقط ينفذ أن لا
ج — الإشراف على الطلاب	45 د	بين سيمر من العرض يوم الأجهزة

ما الذي سنصل إليه في نهاية الدليل

- أنت تعرف لماذا تنجح كل خطوة، لا ماذا تضغط .فقط
- عندك ثلاثة **كسور** مستقلة للأمان نفس :الهدف تزوير التخزين، رتق IL في الذاكرة، رتق دائم للملف — وكسر رابع « نظيف:» مولد .مفاتيح
- عندك **أرقاماً متحققة منها** (لا:استنتاجات كل مفتاح، كل مجموع، كل نمط .بايتات
- عندك **خطة بديلة لكل مرحلة** — والبروفة المسجلة التي تنفذك إن سقط .العرض
- عندك **ردوداً جاهزة** على أسئلة الدكتور والطلاب (الفصل.14)

قبل أن تبدأ • ملاحظة

كل أمر / اختصار مكتوب هنا تمت مطابقته مع توثيق x64dbg .الرسمي **لا يوجد أمر** « من عندي — » وإن ظهر لك سلوك مختلف، فالسبب نسخة الأداة أو توقيت الإيقاف؛ راجع جدول « أخطاءx32dbg الشائعة » في الفصل 4.

لا نستطيع تشغيل MizanPro.exe ولا x32dbg داخل بيئة الكتابة (لينكس،بدون NET. ولا واجهة لذلك كل خطوة في هذا الدليل مكتوبة بصيغة « **تمرين ثم تأكيد** :» جربها مرة واحدة على جهاز العرض، وضع علامة ✓ .بجانها أي خطوة لم تجرب يجب أن تكون لها خطة بديلة .مكتوبة

خريطة الورشة كما فهمت من المشرف

طلب ما	ترجمناه كيفالعرض في الدليل
كل مجموعة تختار برنامجاً أو برمجية خبيثة	محاسبة نظام — Pro ميزان :برنامجنا اخترنا طبقات ثلاث من ترخيص حماية فيه WPF
	شجرة المشروع + README.md

إما كسر الحماية أو تحليل سلوكي	+ (keygen / رتق IL) كسر : الاثنتان ملف + Registry + API سجل (سلوكي (الحالة)	الفصول 6 ، 8 ، 9
عرض 30-40 دقيقة أمام الطلاب	عرض، مقطع 12 في مضغوطة فصلا 14 دقائق 2-4 مقطع كل	الفصل 13
تطبيق مباشر (Live demo)	مفتاح مع « أمامكم يطبق » لحظات ثلاث الرياضيات مثبت	الفصل 7 + 10
الإشراف على مجموعات أخرى تفعل الشيء نفسه	+ منهم نتوقعها أخطاء 5 + مراجعة قائمة تقدير سلم	الفصل 13/3

1. الحماية = فحص + قرار

كل حماية برمجية هي عبارة عن **فحص** ينتج عنه **قرار** (`true/false` أو « اسمح / امتنع »). « الكسر ليس » إزالة الحماية « كما يصوره الناس، بل هو **التحكم في أحد الطرفين** إما أن ترضي الفحص (تولد مفتاحا صحيحا (keygen = ، أو أن تزيف نتيجة الفحص (تعدل الذاكرة / الملف (patch = ، أو أن تقنع طبقة التخزين (تكتب في ال Registry/ الملف ما يوهم بأنك (مرخص

الطبقة	القرار أين	المقابلة الكسر طريقة	صعوبتها
مفتاح تسلسلي محلي	نفس في تحقق دالة الملف	الدالة رتق أو keygen	سهلة جدا
فترة تجربة	عداد + تاريخ مقارنة	المخزن التاريخ تعديل	سهلة
تنشيط مرتبط بالجهاز	مخزنة قيمة / ملف مشفرة	في المفتاح كان إن (التخزين تزوير (البرنامج	متوسطة
تنشيط من خادم	الجهاز خارج	الترخيص شراء إلا — شيء لا	مغلقة عمليا
حصان طروادة / تشفير كامل للوظيفة	شيء من مشتق المفتاح تملكه لا	وقت الذاكرة من المفتاح استخراج التشغيل	صعبة
حماية تجارية (VMProtect/DNA...)	فحوص + افتراضية آلة ذاتية	unpacking + anti-anti-debug	مشروع كامل

القاعدة الذهبية • افحص وتأكد

اسأل دائما « أين تتخذ **Decision** ؟ ». كل كسر ناجح هو جملة: واحدة « القرار يتخذ في الدالة `X ←` سأجعل `X` ترجع ما أريد ».

في ميزان Pro الجواب محدد: وحرفي الدالة `TokenProcessor.ParseAndVerify` ترجع `bool` ، والرتق الذي يجعلها ترجع `1` دائما = كسر. كامل

1. ٢ قاموس 90 ثانية

برنامج كتب ليكسر — هدف تعليمي، لا جريمة. فيه

Crackme

برنامج يشتق مفتاحا صالحا بنفس الخوارزمية لا يغير الملف المستهدف .إطلاقا

تعديل بايتات (في الملف أو في الذاكرة لتغيير منطق .القرار

أداة تطبق الرتق على نسخة الهدف .وتحفظها

تذكير « انتهت التجربة » ؛ إزالته تسمى أحيانا nag-slapping.

بصمة تشتق من العتاد لربط الترخيص بجهاز .واحد

لغة الوسيط في — .NET. الأوامر « المصغرة » التي يولدها المترجم C#.

المترجم :للحظي يحول IL إلى كود آلة عند أول تنفيذ (أو(الثاني للطريقة.

عنوان نسبي داخل الصورة / إزاحة داخل الملف — الجسر بين ما تراه في الذاكرة وما تراه في المحرر Hex.

لماذا نهتم بـ .NET. أصلا؟ • ملاحظة

لأن نصف « أساليب الكسر » الكلاسيكية (التي ولدت في زمن DOS/Win32 و asm (الخام تعمل هنا **بطبقة** :ترجمة بدل أن تعدل `mov eax, 1` تعدل `ldc.i4.1` ، وبدل أن تبحث عن الكود في `.text` تبحث في تيار `#Text` عن **محتوى جسم** .**الطريقة** و `x32dbg` يرى الطبقة السفلى فقط — وهذا بالضبط ما سنجعله نقطة قوتنا لا نقطة ضعفنا (الفصل.7)

١. ٣ الإطار القانوني والأخلاقي (قلهبصوت عال غدا

- المشروعية تأتي من **ملكية** :الهدف البرنامج من بنائنا، الغرض تعليمي، ولا توزيع لأدوات كسر لأطراف .ثالثة
- لا نستخدم ولا ننشر أي شيء يخص برامج تجاريا محمية؛ وإن ذكر طالب مثالا واقعيًا، نحوله فورًا إلى « كيف ندافع؟ ».
- في التقرير النهائي نكتب :صراحة الهدف **MizanPro** crackme ، والغرض تعليمي، ولا آثار على أنظمة .الغير
- الأخلاقي الأهم في :مادتنا **كل ثغرة نشرحها = إصلاح** .نقترحه لا يوجد « هك وامش — » انظر الفصل 12.

٢. ١ :طبقتان ملف التنفيذ والكود المدار

MizanPro.exe (win-x86) الذي تشغله ليس هو البرنامج — هو **apphost** قشرة صغيرة مهمتها تحميل **hostfxr** ثم **hostpolicy** ثم **coreclr.dll** (نواة CoreCLR) وتمرير مسار ال DLL. المدار **منطقك كله يعيش في** **MizanPro.dll** (عندالنشر بدون ملف (واحد داخل قسم **.text** في تيار بيانات اسمه **#Text** (IL + metadata).

الكيان	هو ما	x32dbg يراه ما	dnSpy يراه ما
MizanPro.exe	أصلي apphost بت 32-	ممكنة توقف نقاط + آلة كود (لمصلحتنا معنى بلا)	مضمنا IL يعرض لا
coreclr.dll / clrjit.dll	+ .NET. نواة للحظي المترجم	في «غرباء» الإطارات آلاف المكدس	—
MizanPro.dll	المدارة تجميعتك (IL)	+ الذاكرة خريطة في منطقة #Text تيار	أنواع، الكاملة الشجرة C# ، IL طرق،
حزمة أحادية الملف	سبق ما كل exe داخل مضغوط	القرص على MizanPro.dll لا	٢. انظر — الفتح يفشل ٣

٢. ٢ من C# إلى IL إلى آلة — ولماذا « التوقيت » يحكم

الهدف ثلاثة بايتات فقط 1F 49 D0 —

```
// (السطر - Core/Engine/TokenProcessor.cs المصدر)
return sum % 73 == 0 && xor == RequiredXor;

// (الجزء هذا قراءة فقط منكم المطلوب - مبسط) dnSpy في سيظهر الذي IL
IL_00b1: ldc.i4.s 73 ; 1F 49 ← 73 الثابت حمل
IL_00b3: rem ; D0 ← sum % 73
IL_00b4: brfalse.s IL_00c4 ; 2C 0E ← «false» إلى اقفز 0 ≠ الباقي كان إن
IL_00b6: ... ; ← XOR ال حلقة
IL_00c0: ldc.i4.s 10 ; 1F 0A ← RequiredXor (const → مضمن)
IL_00c2: bne.un.s IL_00c4 ; 39 FC ← «false» إلى اقفز اختلف إن
IL_00c4: ...
IL_00c5: ldc.i4.1 / ldc.i4.0 ; 17 / 16 ← المرجعة القيمة
IL_00c6: ret ; 2A
```

لاحظ أن RequiredXor معرف `const byte` — لذا **يضمن** في IL ولا يولد `ldsflld` يعني ذلك أن البت 0A قابل للتعديل هو الآخر، لكننا سنختار الهدف 73 لأنه (أفريد في الملف، (ب)تغييره إلى 1 يجعل الشرط حسابيا دائما الصدق بدون أن يلمس أي شيء. آخر

أهم جملة في الدليل كله . خطأ شائع — تجنبه

CoreCLR لا يترجم الطريقة إلا عند أول استدعاء لها (وللمرة الأولى يترجمها المترجم السريع `tier0` ، ثم قد يعيد ترجمتها `tier1`).(لاحقا النتيجة :العملية تعديل صورة IL في الذاكرة **بعد** أن تكون `ParseAndVerify` قد تعمل مرة واحدة = **لا شيء يتغير** — لأن كود الآلة الخاص بها ولد من نسخة IL القديمة. لهذا في الفصل 7 سنرتق **قبل** أول محاولة تفعيل، وسنشرح للطلاب لماذا تجرب هذه التجربة مرتين (قبل/ (بعد — فهي وحدها تشرح معنى JIT.

٢. ٣ إعداد النشر الذي اخترناه — ولماذا

الحزمة التي سنحلها

```
<!-- MizanPro.csproj - للورشة النهائية التوصية -->
<RuntimeIdentifier>win-x86</RuntimeIdentifier>
<SelfContained>true</SelfContained> <!-- جهازا 20 على .NET Runtime. تثبيت نريد لا :ابقها -->
<PublishSingleFile>false</PublishSingleFile> <!-- الورشة ينقذ سطر هذا :true من غيرها -->
<DebugType>none</DebugType> <!-- «حقيقي» برنامج من يقربنا :PDB لا -->
<Optimize>false</Optimize> <!-- dnSpy في للقراءة أسهل :خام IL -->

:: البناء

dotnet publish -c Release -r win-x86 --self-contained true ^
-p:PublishSingleFile=false -o dist
```

بعد البناء يجب أن ترى `dist\MizanPro.dll` — إن لم تره فالتغيير لم يطبق، وأوقف كل شيء

الإعداد	(الحالي) <code>SingleFile=true</code> (مطلوب)	مع <code>SingleFile=false</code>
ما هو على القرص	<code>MizanPro.exe</code> واحد ~150MB	<code>MizanPro.exe</code> + صغير <code>MizanPro.dll</code> + مكتبة ~250
هل يفتح dnSpy الهدف؟	للتحليل مدعومة غيرالحزمة (لا (الحفظ /	<code>MizanPro.dll</code> يفتح — نعم IL ويعرض
«Save Module» بعد التعديل	تقنيا مستحيل	معدلا ملفا وينتج يعمل
مكتبات <code>System.*</code> لمرجعيات dnSpy	الحزمة داخل	C# محرر ← DLL بجانب حلها يستطيع
ما يراه المنقح في خريطة الذاكرة	<code>MizanPro.exe</code> في الحزمة فقط	(<code>MizanPro.dll</code> باسم منطقة (فيهابحث

هل يحتاج الطالب تثبيت Runtime ؟	لا (self-contained)	لا (self-contained)
---------------------------------	---------------------	---------------------

لا تنس التبعات • تنبيه

Workshop_Pack.bat ينسخ ملفا واحدا الآن — يجب أن ينسخ محتويات dist\ كاملة، و LabSheet.md / README_Workshop.md يجب أن تطلب تشغيل MizanPro\MizanPro.exe لا « ملفا واحدا. » وإلا سيستلم الطلاب ملفا لا يعمل في إصدار 32- بت من apphost ستجد coreclr.dll (وليس clr.dll (القديم في خريطة الذاكرة — لا تكتب clr.dll على السبورة

٢. ٤ متغيرات بيئة نثبتها لنرى IL بوضوح

• cmd ضبط البيئة للبروفة

:: قبل تشغيل x32dbg (نافذة cmd واحدة تكفي للجلسة (كلها

```
set DOTNET_TieredCompilation=0 :: الجلسة أثناء تغيير لا الأصلي الكود عناوين ← tier1 ترجمة إعادة لا
set DOTNET_ReadyToRun=0 :: وقتها في تترجم طريقة كل ← الحزمة داخل مسبق أصلي كود لا
set DOTNET_gcServer=0 :: Threads view في «مفاجآت» أقل) معتاد GC سلوك ::
```

:: اختياري إظهار معلومات JIT في Log

```
set DOTNET_JitDisasm=ParseAndVerify :: فقط للمتقدمين؛ build مع DIAGNOSTICS يحتاج ::
```

السطران الأولان هما: المهمان يجعلان « ماتراه في الذاكرة = ما ستراه بعد لحظات — » وشرط أي شرح على مستوى كود الالة

٣. قائمة التثبيت (افعلها(الليلة

الأداة	نأخذ ماذا	خطتنا في لماذا	تعذر إن بديل
· x64dbg (v50+) x32dbg.exe	→ الرسمي ZIP ثم استخرجه شغله	+ ذاكرة + breakpoints :النجم IL رتق + سجلات	OllyDbg2 (محدود) — ينصح لا
dnSpyEx	من إصدار آخر المستودع	الطريقة جسم تعديل IL/ قراءة الملف وحفظ	ILSpy (فقطقراءة + Some edit)
HxD	الرسمي الموقع	(القرص على بايتات تديل / بحث MizanPro.dll)	hex محرر أي
PE-bear أو CFF Explorer	المستودع	والربط #Text / الأقسام رؤية ملف RVA↔	من يدوي حساب الأقسام جدول
Procmon	Sysinternals	فعلا التطبيق يكتبماذا « دليل (ملفات / Registry)	رسائل x32dbg log-only
Python 3 + pycryptodome	pip install py cryptodome	تزوير state.enc (AES-CBC + Base64)	C# console صغير
OBS Studio	الرسمي الموقع	العرض ينقذ الذي الاحتياطي الفيديو	Xbox Game Bar

ثلاث أخطاء تضيع ساعة قبل العرض · خطأ شائع — تجنبه

تشغيل x32dbg.exe من داخل ال ZIP بدون استخراج. «missing dll / app crash» → استخراج
المجلد. كاملا
فتح x64dbg.exe لهدف win-x86 → لن يرى ما تتوقعه (أو).يرفض هدفنا 32- بت ←
x32dbg.exe.
مضاد فيروسات يحذف مجلد x64dbg أو MizanPro.exe → (packer/behavior) استثن المجلد
C:\Lab\ وأوقف الحماية اللحظية أثناء البروفة. فقط

٣. ٢ مجلدات مرتبة (هيكل،لا (عشوائيا

```

C:\Lab\
MizanPro\      ← dist\ كاملا (MizanPro.exe + MizanPro.dll + المكتبات)
x32dbg\Release\ ← المستخرج
tools\         ← HxD.exe, dnSpy.exe, PE-bear.exe, Procmon64.exe
keygen.py      ← (الغد قبل يعمل جربه) 10 الفصل من
forge_state.py ← 6 الفصل من
reset-license.bat ← ٣.٤ من
shots\        ← (العرض جهاز دقة نفس) العرض لقطات
backup\       ← MizanPro.dll النظيف + MizanPro_patched.dll

```

سبب الترتيب ليس: جماليا كل مسار طويل تكتبه أثناء العرض هو ثانية صمت أمام 60 طالبا، وكل ملف « على سطح المكتب » يعني أنك ستجد نفسك تبحث في قائمة الإكمال التلقائي عن `C:\Users\شمه\Desktop\....`

٣. إعدادات x64dbg الافتراضية (افعلها مرة واحدة) للأبد

- **Options → Preferences → Events:** فعل **System breakpoint** (نقطة توقف عند تحميل **ntdll** — هنا تكون ال DLLs الرئيسية (محملة ، وأبق **Entry point** مفعلا. أيضا
- **Options → Preferences → Exceptions:** أضف **0xE0434352** (CLR first-chance) إلى القائمة المتجاهلة — وإلا سيتوقف المنقح كل 3 ثوان بسبب استثناءات CLR. الداخلية
- **Options → Preferences → Appearance:** حجم الخط 12-14 لا Disassembly و 9-10 لا Log (البروجكتور » يأكل » الحروف). (الصغيرة
- **Options → Preferences → General:** عطل **Confirm on exit** حتى لا يضيع منك توقيت الانتقال بين. الشاشات
- احفظ الإعدادات (**dbsave** للتعليقات / العلامات داخل جلسة الهدف، و **Config** للأداة (نفسها وانسخ المجلد إلى جهاز. الاحتياط

٣. ٤ التنظيف قبل كل تجربة (script واحد)

```
@echo off
REM reset-license.bat — يعيد ميزان Pro إلى « نسخة جديدة على هذا الجهاز »
reg delete "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /f >nul 2>&1
reg delete "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /f >nul 2>&1
reg delete "HKCU\Software\MizanSolutions\Mizan\Settings" /f >nul 2>&1
del /f /q "%APPDATA%\MizanPro\state.enc" 2>nul
echo MizanPro license state cleared.
pause
```

ملاحظة: 32/64-bit HKCU\Software غير محول (لا يوجد Wow6432Node لا — HKCU) القيمة التي يكتبها تطبيق 32- بت تراها regedit في نفس المكان.

ثلاث حالات نحتاجها في العرض

cmd • ثلاث حالات جاهزة

:: (أنسخة جديدة تماما

reset-license.bat

:: («ب» انتهت التجربة — « لإظهار نافذة الانتهاء) الفصل(8

reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /t REG_SZ /d 20250101 /f

:: («ج» مفعّل جاهز — « للقطات وللبدايات السريعة

reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /t REG_SZ /d M1Z4N-2025P-ROM4S-T3R01 /f

٤. ١ ما الذي تراه عند أول F9

بعد **File → Open (F3)** واختيار **MizanPro.exe** يتوقف x32dbg عند **System Breakpoint:** أنت داخل **ntdll** ، قبل أن يصل الكود إلى **main** ، والصورة المحملة حتى الآن. قليلة اضغط **F9** لتتمة الإقلاع؛ ستمر بنقطة **Entry point** ، ثم تظهر نافذة البرنامج **هنا نريد أن نكون قد وصلنا** إلى **MizanPro.dll** أصبح في الذاكرة، لكن طرائقنا لم تنفذ. بعد

اللوحه	فتحتها اختصار	ورشتنا في نفعها ما
CPU / Disassembly	Alt+C	للتعليق ; + المحلي للتعديل Space + الأصلية الأوامر
Registers	—	الأعلام ورؤية (8 الفصل تمرين) eax تعديل
Call Stack	Alt+K	كودك من لا ، coreclr / JIT من جاء النداء أن إثبات
Memory map	Alt+M	7 الفصل بوابة — MizanPro.dll منطقة على العثور
Breakpoints	Alt+B	(bc) حذف / واحدة مرة / سجل / شرط
Log	Alt+L	الجمهور لإقناع لوحة أهم — « ير لم / رأى » دليل
Patches	Ctrl+P	الجلسة هذه في رتقته ما كل مراجعة
Notes	Alt+N	dbsave مع تحفظ — العناوين فوق ملاحظاتك

٤. ٢ الأوامر التي سنستعملها فعلاً (ومعناها)الحرفي

أمر / مفتاح	يفعله ما	نستخدمه متى
F9	متابعة / تشغيل	توقف نقطة كل بعد
F7 / F8	فوق خطوة / داخل خطوة	مترجم IL داخل سطر - سطر المشي
Ctrl+F9	الطريقة من (ret) العودة حتى تنفيذ الحالية	eax التقاط لحظة
Alt+F9	« المستخدم كود » حتى تشغيل	coreclr / ntdll زحام من للخروج

F4	المحدد السطر حتى تشغيل	إلى تابع فقط توقف، نقطة أريدلا « هنا »
F12	(pause) مؤقت إيقاف	في البحث قبل النافذة، ظهور بعد الذاكرة
F2	الحالي السطر على توقف نقطة تبديل	مثلا <code>bp user32.MessageBoxW</code> على
Ctrl+G	اسم / عنوان إلى اذهب	أو <code>user32.MessageBoxW</code> <code>MizanPro.dll+1234</code>
Ctrl+F / Ctrl+B / Ctrl+E	ثنائي بحث / نصي بحث ثنائي تعديل / (pattern)	<code>1F 49 D0</code> نمط : 7 الفصل
Space	السطر على Assemble نافذة فتح المحدد	<code>ldc.i4.s 1</code> يبدو كيف لتسجيل
;/:	(label) تسمية / تعليق إضافة	شاشتك الجمهور يفهم حتى
Ctrl+D / Ctrl+Backspace	للتحديد الرجوع / مرجعية علامة السابق	الطلاب أمام السريع التنقل
Ctrl+F2 / Alt+F2	إغلاقه / الهدف تشغيل إعادة	بسرعة « بعد / قبل » لتجربة
<code>bp, bc, bph, bpm</code>	/ الكل مسح / عادية توقف نقاط ذاكرة كتابة على / نداء على	٣. ٤. انظر
<code>log, msg, find, set</code>	في بحث / رسالة / Log في كتابة قيمة تعديل / الذاكرة	بصريا يوثق شيء كل

أوامر لا توجد في x64dbg · خطأ شائع — تجنبه

`bpdll` (موجود في `CrackGuide.md` — (احذفه ، `bpx` / `bpm` بأسلوب OllyDbg المعطوف،
`Run till return` كسطر أوامر (`ret` الاسد `/Ctrl+F9`).
الكتابة الصحيحة في شريط الأوامر `bp user32.MessageBoxW` — ؛إل في آخره .لتعليق أي
أمر آخر سيظهر `unknown command` أمام الجمهور

٤. ٣ نقاط :التوقف الأنواع الأربعة التي ستحتاجها

- **Software breakpoint (F2 / `bp`)**: يستبدل `int3` (0xCC). مؤقتا هو الافتراضي، وهو كاف لكل ورشتنا
- **Conditional**: كليك يمين على ال ← Edit ← breakpoint `Condition = eax==1` — لتتوقف فقط عند الشرط المهم (:مثلا عند أول رسالة تحتوي « انتهت » بدل كل .رسالة)
- **Log-only (الأناقة:الحقيقية)** `Log text = {p:arg1}` + `Break = Never` + `Command = gc` ← (go and count) يصب نصوص الرسائل في `Alt+L` دون أن يوقف

البرنامج — أقوى لحظة « سلوكية » يمكنك عرضها

- **Hardware / memory** (`bph`, `bpm`): على كتابة / قراءة .عنوان مفيد في native ، غير مناسب لمتغير مدار في .NET (عنوانه يتغير مع GC لا تقع فيه .غدا

DLL لم تحمل بعد؟ • ملاحظة

إذا أردت نقطة توقف على `user32.dll!MessageBoxW` في بداية الجلسة، لا تحتاج أمرا سحريا اذهب إلى ``Break on DLL load`` → **Options** → **Preferences** → **Events** ، تابع حتى يحمل الـ DLL (ستراه في Log) ، ثم عطل الخيار واضبط الـ `bp` :عمليا `user32` محمل أصلا عند System Breakpoint في تطبيقات WPF ، فغالبا لن تحتاج ذلك .الأسلوب الأبسط للهدف :المدار لا تنتظر — DLL انتظر أول نقرة (الفصل.7)

٤. ٤ خريطة الذاكرة والبحث الثنائي (مهارتك الأساسية (غدا

- 1 بعد ظهور نافذة التطبيق اضغط **F12** (إيقاف.(مؤقت ستجد RIP داخل `ntdll` — لا يهم
- 2 → **Alt+M** (Memory map) ابحث في العمود «Module» عن `MizanPro.dll` → **Follow → In Disassembler** (أوفي .Dump) ←
- 3 في نافذة :الذاكرة **Ctrl+B** (Binary search) واكتب `1F 49 D0` ، وحدد النطاق «Current module» إن طلب منك، ثم **Find all**.
- 4 انقر نتيجتك، تحقق من البايت الذي يليها (`=2C`) `brfalse` لتعرف أنك في المكان الصحيح (الفصل.٥. ٢)
- 5 **Ctrl+E** يفتح محرر الذاكرة الثنائية ← عدل البت `49` إلى `01` .OK. افتح **Ctrl+P** لترى الرتق .مسجلا
- 6 **F9** تابع، ولا تغلق x32dbg بعد التجربة — **Patches** تظهر لك « ماغيرته » في حالة أردت إعادته.

كيف تعرف أن الكتابة في الذاكرة ستقبلها Windows ؟ • افحص وتأكد

صورة الـ DLL محملة بخصائص `write-copy` ؛ أول كتابة **تنسخ الصفحة** ثم تعدل النسخة الخاصة .بالعملية :لذلك ((الن ترى `Access violation` ، ((٢)الملف على القرص لا يتغير ، ((٣)بعد `Detach` أو إعادة التشغيل تعود الأمور كما .كانت هذه بالضبط خواص « الرتق المؤقت » التي سنستغلها في الفصل 7.

٤. ٥ أخطاء x32dbg الشائعة — جدول إنقاذ

العلاج

التشخيص

الشاشة على العرض

يتوقف ولا يشتغل البرنامج IL نقطة عند أبدا	ال ضبط قبل ترجمت الطريقة breakpoint (JIT)	اضبط ثم (Ctrl+F2) التشغيل إعادة IL رتق أو الاستخدام؛ قبل النقطة 7 الفصل في كما
توقف / Access violation ثانية كل ntdll داخل	CLR first-chance استثناءات	في 0xE0434352 تجاهل Exceptions
unknown command	OllyDbg/Windbg من أمر	٢. ٤. راجع
في MizanPro.dll يظهر لا Memory map	يوجد لا (الملف أحادي النشر بعد يحمل لم أو DLL)	F9 ب تابع أو النشر؛ إعداد غير بالتحميل يخبرك Log حتى
لبحث كثيرة النتائج 1F 49 D0	/ تواريخ في يظهر قصير نمط أخرى جداول	وحدد ، 1F 49 D0 2C ابحث موديول داخل النطاق MizanPro.dll
??? تعرض Disassembly ال غريبة أكواد أو	لا JIT من مولد كود في أنت IL ك تفكيكه يمكن	من IL تشرح لا — طبيعي هذا التعليمي للعرض dnSpy افتح هنا؛
bad copy of the executable F3 عند e	AV أو مقفول / مضغوط ملف تدخل	مستثنى، مجلد في الهدف استخرج للحظية الحماية وأوقف
في Unicode text ال أرى لا الذاكرة	يظهر لا معرب UTF-16 نص في «ASCII string»	ثم Follow in Dump استخدم بباينات find أو — «Unicode» UTF-16
Detach بعد الجلسة سقطت	حالات بعض يحب لا CLR الإنهاء	x32dbg (Alt+F2) من العملية أغلق يدويا تشغيلها أعد ثم
Load ... ب ممتلى Log ال جديد ولا	« يصرخ Log وضع في أنت مع DLL load breakpoints	وامسح Break on DLL load عطل Log (→ يمين كليك) Clear

٥. ١ كيف يفكر من كتب الحماية

الفكرة في **README.md**: واضحة « الترخيص حالة، وليس شاشة. » التطبيق لا يسأل المستخدم كل مرة، بل يقرأ حالة **مخزنة**. ويصدقها لذلك الطبقات الثلاث كلها: **محلية** لا خادم، لا توقيع رقمي، لا مفتاح خاص. مخفي

المخطط الحقيقي كما هو في الكود

[1] `TokenProcessor.ParseAndVerify(rawToken) : bool` → الترخيص مفتاح

شكل: `XXXXX-XXXXX-XXXXX-XXXXX` من `[A-Z0-9]`

القبول: `(Σ ASCII % 73 == 0) && (XOR == 0x0A)`

حاسوب أي على يعمل صالح مفتاح أي - إطلاقا بالجهاز يرتبط لا ▲

[2] `ProductStateEngine.EvaluateCurrentState() : bool` → المخزنة الحالة

أ) `HKCU\Software\MizanSolutions\Mizan\RuntimeState` → `ActivationBlob`

(إضافي تحقق أي بدون نعم،) `PRO` ← صالحا المخزن المفتاح كان إن

(«اليدوي التعديل كشف» آلية) يحذف ← صالح غير لكنه موجودا كان إن

ب) `%AppData%\MizanPro\state.enc = Base64( AES-128-CBC( "MIZAN_VALID|<الجهاز بصمة>" ) )`

`PRO` إلى يرقى ولا) بالتشغيل يسمح ← البصمة + العلامة `**يحتوي**` أن يجب محتواه

ج) `InstallDate = yyyyMMdd` ← أيام 7 + تجربة، مرة تقرأ واحدة عند الإقلاع

[3] الترخيص → `AuthWindow / Controls/ActivationPanel / SettingsPage`

المفتاح الثابت `Mizan2024_K3y128` ومتجهه `MizanPro_IV_2024` مكتوبان نصا في `ProductStateEngine.cs` (السطران 59 و 62) « تشفير » بمفتاح داخل الملف ليس تشفيراً من وجهة نظر أمنية، وهذا موضوع الفصل 12.

٥. ٢ الدالة التي نريدها — بكاملها

```

public static bool ParseAndVerify(string rawToken)
{
    if (string.IsNullOrEmpty(rawToken)) return false;
    string clean = rawToken.Replace("-", "").ToUpperInvariant();
    if (clean.Length != 20) return false;
    foreach (char c in clean)
        if (!((c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9'))) return false;

    int sum = clean.Sum(c => (int)c);
    byte xor = 0;
    foreach (char c in clean) xor ^= (byte)c;

    return sum % 73 == 0 && xor == RequiredXor;    // نقف هنا ←
}
private const byte RequiredXor = 0x0A;           // IL (const) في مضمن

```

ملاحظة مهمة: للدقة `sum` و `xor` لا يعتمدان على أي شيء يخص الجهاز، ولا يوجد « ملفسري » ولا وقت صلاحية في المفتاح — لهذا الـ `keygen` ممكن (الفصل 10).

٥. ٣ الأهداف الثلاثة للرتق (اخترواحدا وافهمم (الباقى

النتيجة	الرتق معنى	بايتات	عنه تبحث ما
صحيحة وأبجدية 20 بطول مفتاح أي محرفا 20 مفتاح أي :لا ... ومجموع ، مجموعه كان مهما صحيحة بصيغة مطلوبا زال ما XOR وال	<code>sum % 1 == 0</code> صادق دائما	1F 49 D0 → 1F 01 D0	<code>ldc.i4.s 73 +</code> <code>rem</code>
كسر نصف — XOR (0x0A) فحص يبقى	فحص تخطي المجموع	2C xx → 90 90	بعد <code>brfalse.s</code> <code>rem</code>
بفحصي يمر مفتاح لكل <code>return true</code> كامل كسر — والأبجدية الطول	إطلاقا قفز لا	2C xx → 90 90 و 39 xx → 90 90	(القفز كلافرعي <code>brfalse+</code> <code>bne.un)</code>
نستخدمه لن — فقط «الذوق» يغير يتكرر قد <code>1F 0A</code> نمط	0 مع القارنة	1F 0A → 1F 00	0x0A مع المقارنة

لماذا اخترنا السطر الأول • تنبيه

1F 49 D0 نادر في الملف (الثابت 73 (نادر ، والتغيير بحجم متطابق) بايت واحد لا يتغير معه أي طول / إزاحة في جدول ال (metadata ، والشرط يصبح صادقا رياضيا لا بقفزة تحكم — أي أنه أوضح .تعليميا

تحذير :الدقة بعد الرتق الأول يبقى فحص `xor == 0x0A` معمولا به — لذلك سنستخدم في العرض مفتاحا طوله صحيح، أبجديته صحيحة، XOR صحيح، ومجموعه لا يقبل القسمة على 73. هذا هو « المفتاح المثبت للرتق ».

5. الأرقام التي سنقولها على المسرح (كلها متحقق منها)

المفتاح Σ المجموع 73 %	XOR	الرتق قبل	رتق بعد
M1Z4N-2025P-ROM4S-T3R01	0 ✓	0x0A ✓	مقبول
AAAAABBBBBBBBBCCCCCADN89	0 ✓	0x0A ✓	مقبول
5NOFK-JQB1Z-7HSHF-NOP6D	39 X	0x0A ✓	الدليل ← مقبول مرفوض
MIZAN-2026X-32DBG-B36SQ	0 ✓	0x0A ✓	مقبول
26F30-20C2B-22E4B-B (!جهاز بصمة)	—	(مرفوض (16الطول	مرفوض

- $1426 = 73 \times 19 + 39$ — لا يقبل القسمة ⇒ مرفوض؛ ومع أن `XOR = 0x0A` صحيح ⇒ الفشل من المجموع وحده لهذا المفتاح واحد مثالي لتجربة .الرتق
- الصف الأخير :مهم في لقطة [the problem/215449.png](https://the-problem.com/215449.png) لصقت بصمة الجهاز في حقل المفتاح، فأعاد المنسق تقسيمها إلى `26F30-20C2B-22E4B-B` ⇒ (5-5-5-1) رسالة « مفتاح غير صالح » صحيحة لكنها **مضللة للعين** — سنشرحها كدرس UX/ أمان (الفصل.11)
- كل المفاتيح في الجدول ولدت / فحصت بسكرت الفصل 10 نفسه — لا يوجد رقم « تقريبي » في هذا .الدليل

هذه أسرع كسرة في المشروع، وأقربها إلى ما يفعله العالم الحقيقي حين يبيع « ترخيصاً مدمى الحياة مسرباً: » لا نلمس البرنامج إطلاقاً — نكتب ما يتوقع البرنامج. قراءته

٦.١ الكسرة (): (ActivationBlob = مفتاح صالح

- 1 أغلق التطبيق، وشغل `reset-license.bat` (الفصل ٣. ٤).
- 2 `reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /t REG_SZ /d M1Z4N-2025P-ROM4S-T3R01 /f`
- 3 شغل `MizanPro.exe` → سجل الدخول بـ `admin` / `Admin@123.`
- 4 النتيجة: المتوقعة لا نافذة « تفعيل » ، اللافتة تقول **PRO** ، وصفحة الإعدادات « الترخيص تعرض المفتاح. نفسه
- 5 **الدليل الذي يقنع** احذف القيمة وكرر — ستعود لافتة. التجربة الفرق الوحيد هو قيمة في الـ Registry.

لماذا ينجح هذا؟ (الجواب الذي يسأل عنه (المشرف . خطأ شائع — تجنبه

`EvaluateCurrentState` يقرأ القيمة ثم يمررها إلى `ParseAndVerify` — أي أن معيار الثقة هو صحة الصيغة الحسابية وحدها ، لا توقيع ولا ارتباط. بالجهاز البرنامج « يحمي » بمفتاح يمكن لأي أحد حساب صفاته، ثم يخزن النتيجة في مكان يكتبه المستخدم. نفسه هذا ليس خلافاً في `regedit` — هذا: تصميم لا يوجد سبب يجعل القيمة المخزنة غير قابلة. للتزوير (الإصلاح) الفصل: (١٢) اجعل القيمة المخزنة نتيجة توقيع بمفتاح لا يملكه المستخدم (أو اجعل القرار عند (الخادم

٦.٢ الكسرة (): (state.enc ونحن لا نملك (!المفتاح

هنا الجملة: التعليمية « ملف مشفر بمفتاح AES يبدو منيعاً — إلا أن المفتاح داخل الملف نفسه و `static readonly` سنعيد إنتاج الـ ciphertext بحرية، والبرنامج سيقبله كمسار احتياطي. صالح

```

# forge_state.py — يحتاج pip install pycryptodome
# الاستخدام < python forge_state.py _ البصمة _ التي _ يعرضها _ التطبيق >

import base64, os, sys
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

KEY = b"Mizan2024_K3y128"          # ProductStateEngine.cs:59
IV  = b"MizanPro_IV_2024"         # ProductStateEngine.cs:62
fp  = sys.argv[1].strip().upper() # مثل: 1A2B-3C4D-5E6F-7A8B (التفعيل شاشة من انسخه)
plain = f"MIZAN_VALID|{fp}".encode()
ct = AES.new(KEY, AES.CBC, IV).encrypt(pad(plain, 16))
path = os.path.expandvars(r"%APPDATA%\MizanPro\state.enc")
os.makedirs(os.path.dirname(path), exist_ok=True)
open(path, "w").write(base64.b64encode(ct).decode())
print("written:", path)

```

لا تحسب البصمة بنفسك — `GetFirstMacAddress` يعتمد WMI وترتيب المحولات، والاختلاف حرف واحد يبطل الملف **انسخها من الشاشة** (**معرف الجهاز** أو من صفحة الإعدادات).

- جربها مع `InstallDate` قديماً (الحالة) (بفي ٣. ٤) → التطبيق كان سيرفض «) انتهت التجربة ← «) بعد كتابة `state.enc` **يعمل** رغم انتهاء: التجربة المسار الاحتياطي يسبق فحص التاريخ.
- لاحظ أنه **لا** يرقى إلى PRO (المسار » صامت — «) لا تبالغ في النتيجة أمام الجمهور؛ قل بالضبط ما يفعله
- هذا مثال ممتاز لـ « الحماية نصف مكتملة: » المطور احتاط لفقدان ال Registry ، فنسف فعالية الحماية. كلها

٦. ٣ الكسرة (:جإعادة ضبط التجربة) كلاسيكية ال (trial reset)

cmd . « ركباليوم — « سبعة أيام جديدة

```
reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /t REG_SZ /d %date:~10,4%%date:~4,2%%date:~7,2% /f
```

△ لا تدخل هذا في العرض كأساس: `TrialPeriodDays` تقرأ مرة واحدة عند الإقلاع ولا تتحدث أثناء الجلسة، ولأن `EvaluateCurrentState` يعمل داخل نافذة Splash قبل أن تتمكن من إيقاف المنقح — سنستخدمه فقط « تمهيداً» للفصل 8.

٦. ٤ ما الذي يثبت أن التطبيق « لم يكتب شيئاً » عند المفتاح الخاطئ

سؤال يطرحه الطلاب: دائماً « كيف نعرف أن الفشل فشل حقيقي؟ ». نفحص بـ Procmon أو x32dbg log-only:

```
$ bp advapi32.RegSetValueExW          + Log text = {p:arg2}
قيمة أي تكتب لم ⇒ توقف لا → خاطئ مفتاح أدخل ثم
كتبت ⇒ ActivationBlob الاسم عند يتوقف → صحيح مفتاح أدخل ثم
(Break = Never + Command = gc) breakpoint → ال على يمين كليك :الرسالة تظهر لم إن)
```

البدل بلا :أدوات `reg query "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob` قبل وبعد — إن ظهر `error: the system cannot find the file specified` ثم بعد التفعيل ظهرت القيمة، فهذا دليل. مكتمل

لماذا هذا الكسر هو « الأصغروا أنظف » ؟ • ملاحظة

لا يغير أي ملف (فلا Anti-Virus ، ولا فحص تكامل، ولا حاجة لإعادة (تنشيط ، ولا يحتاج حتى مفتاحا « مشتقا- » بل يجعل دالة التحقق نفسها عاجزة عن الرفض هذا هو تعريف patch في أدبيات: الكراك « تعديل القران، لا الإدخال. »

v. القاعدة الزمنية (اقرأها)مرتين

ParseAndVerify لا تحتوي كود آلة حتى تستدعى أول مرة بعدها، تعديل صورة IL في الذاكرة لا معنى له: الكود المنفذ هو نسخة مترجمة. قديمة لهذا الترتيب هو كل شيء رتق قبل أول محاولة. تفعيل

التسلسل	يحدد 5N0FK-JQB1Z-7HSHF-NOP6D إدخال عند النتيجة
← F12 ← النافذة ظهور ← تشغيل التفعيل محاولة ثم ← رتق	أن قبل رتق IL يترجم
(فشل) المفتاح إدخال ← تشغيل المحاولة إعادة ← رتق ← F12 ←	من ترجمت الطريقة أولا الأصلية IL
ثم (HxD) القرص على رتق تشغيل	جديدة، كلها الحزمة قديمة ذاكرة لا

v. الخطوات — حرفيا كما سنضغطها

- 1 تأكد من الحزمة: الصحيحة `dir C:\Lab\MizanPro\MizanPro.dll` يجب أن يوجد إن لم يوجد فإعداد النشر لم يتغير — أوقف كل شيء وعد للفصل ٢. ٣.
- 2 افتح `x32dbg.exe` (مسار-32 بت من `C:\Lab\x32dbg\Release\` ، واضبط متغيرات البيئة في نفس نافذة cmd التي ستطلق منها إن أردت `DOTNET_TieredCompilation=0` .
- 3 `File → Open (F3)` واختر — `MizanPro.exe` لا `MizanPro.dll` ؛ هذه هي الطريقة الوحيدة لتشغيل apphost تحت المنقح.
- 4 — **F9** تجاوز System Breakpoint. تابع بـ **F9** حتى تظهر نافذة تسجيل الدخول (قد تحتاج **F9** مرة أو مرتين حسب `Entry point`).
- 5 **F12** (إيقاف. مؤقت لا تخف من أن RIP في `ntdll` — لا نريد كود الآلة. أصلا
- 6 **Alt+M** لفتح خريطة الذاكرة، وابحث عن سطر ال module باسم `MizanPro.dll` (ستجده قرب الأعلى بعد `MizanPro.exe`).

- 7 علك على ذلك السطر ← كليك يمين ← **Dump / Follow → In Memory Map** : أو ببلا **Ctrl+B** (Binary search) ضمن النطاق «All modules»/ **MizanPro.dll**
- 8 في نافذة البحث اكتب النمط **1F 49 D0** ← **Find all**. **توقع نتيجة واحدة**. إن ظهرت أكثر ابحث عن **1F 49 D0 2C** ، والنتيجة الصحيحة هي التي تليها بايت القفز **2C** وتقع داخل منطقة **.text**. لا **DLL**. المدار
- 9 انقر النتيجة **Ctrl+E** ← (تعديل الذاكرة (الثنائية ← غير البت **49** إلى **01**. **OK** ←) البايتان الآخران **1F** و **D0** كما (هما
- 10 **Ctrl+P** (Patches): تحقق أن السطر يقول **1F 01 D0** — هذه لقطة ممتاز للعرض « رتق واحد، بحجمه الأصلي ».
- 11 **F9**. للمتابعة الآن ادخل **admin** / **Admin@123** ثم افتح « تفعيل المنتج » :والصق **5N0FK-JQB1Z-7HSHF-N0P6D** → **تم !التفعيل**
- 12 **Klik** **يمين في ال** → **Disassembly** لا **تفصل المنقح** — اترك العملية تعمل، ثم بعد العرض أغلقها من **x32dbg** ، وشغل **reset-license.bat** ، وتحقق أن **state.enc** و **ActivationBlob**. حذف الملف على القرص لم يمس ⇒ أي فحص تكامل لن يشتكي

الدليل الكامل أمام الجمهور (ثلاث(لقطات - افحص وتأكد

- (١)قبل: الرتق نفس المفتاح ⇒ « مفتاح الترخيص غير صالح » (نسخة ال Log/ .الرسالة
- (٢)بعد: الرتق نفس المفتاح **بالضبط** ⇒ « تم التفعيل بنجاح + » الالفتة **PRO + reg query ActivationBlob**. يظهر
- (٣)أعد تشغيل التطبيق بدون المنقح ⇒ المفتاح صار **غير صالح** مرة أخرى (الرتق كان في الذاكرة (فقط — هذه أقوى جملة تقنع المشرف أنك تفهم الفرق بين الذاكرة .والملف

٧. ٣ « التجربة الثانية: لماذا لم ينجح الرتق هذه المرة؟

أعد كل شيء، لكن أدخل المفتاح **قبل** ال patch (لتحصل على رسالة (الرفض ، ثم رتق، ثم أعد المحاولة ⇒ **سيرفض** اسأل: الحضور « ما الذي تغير؟ » ثم: أجب الطريقة ترجمت بواسطة JIT إلى كود آلة في الذاكرة، وصار تنفيذها لا يقرأ IL. ثم: واصل « لهذا نرتق على مستوى IL **قبل** الاستخدام الأول — أو نرتق الملف (الفصل ٩. «) هذه اللحظة وحدها تساوي نصف الدرجة

خطة بديلة إن لم تجد النمط (احفظها - تنبيه

(أتأكد أنك في MizanPro.dll لا MizanPro.exe (الأخير حزمة apphost).
(بتأكد من إعداد النشر (PublishSingleFile=false — بدون لا يوجد MizanPro.dll على الإطلاق
(جافتح MizanPro.dll في HxD وابحث عن 1F 49 D0 على القرص؛ إن وجدت هناك ولا تجد في الذاكرة ⇒ أنت في نافذة DLL خطأ في خريطة الذاكرة (هناك MizanPro.ni.dll أو مناطق Image / Heap).
(دإن كان كل شيء صحيحا والنمط: مفقود استخدم 1F 49 D0 2C ، أو ابحث عن D0 2C (rem + brfalse) قرب 1F 0A (الحمل الثابت — 10 واختر الأول
(هآخر: خطة ألغ R2R) المتغير(DOTNET_ReadyToRun=0 وأعد بناء الحزمة بدون crossgen — وراجع أنك لم تبين بملف MizanPro.dll من dist. قديم

v. ٤ النسخة « الأكثر جراءة » (للقوة فقط، لا للضرورة

بدل تغيير 73 إلى 1 ، يمكن تخطي فرعي القفز: معا 2C xx → 90 90 و 39 xx → 90 90 ⇒ الدالة تعود true لكل مفتاح له 20 محرفا من `A-Z0-9` (فحوص الطول / الأبجدية تسبق القفزات، فما زالت. (تعمل مفيد لإظهار أن « الترقيع ليس كلاسيكيا بايت واحد بالضرورة — أحيانا تعليمات قفز » ؛ وأيضا يبقي احتمال أن يسألك أحد « وماذا لو كان المفتاح 21 محرفا؟ ».

هذا هو الجزء الذي يجب أن نسميه باسمه الصحيح أمام :المشرف **تشریح سلوك** ، لا .كسر سنري الطلاب كيف يرى المنقح نداءات Win32 ، وكيف تقرأ قيمة الإرجاع، ثم نشرح بصراحة لماذا لا يغير ذلك شيئاً في — MizanPro وهذا الصدق هو ما يفرق عرضاً هندسياً عن « خدعة».

٨. ١ جهر :الحالة « انتهت التجربة »

· cmd إنهاء التجربة (سطر واحد (يكفي

```
reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /t REG_SZ /d 20250101 /f
del /f /q "%APPDATA%\MizanPro\state.enc"
reg delete "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /f
```

بعد هذا، أول ما يظهر عند الإقلاع صندوق « انتهت فترة التجربة ». ... هنا نريد أن نكون تحت المنقح **قبل** أن يظهر.

٨. ٢ الخطوات

- 1 شغل **MizanPro.exe** تحت x32dbg (F3) ثم **F9** حتى System Breakpoint.
- 2 في شريط الأوامر **bp user32.MessageBoxW** ثم **F9** (**user32.dll** محمل أصلاً في تطبيقات WPF ؛ إن قال **module not found** تابع **F9** حتى يظهر في Log ثم أعد الأمر وقاية :إضافية (**bp user32.MessageBoxIndirectW**).
- 3 — **F9** سيتوقف عند أول صندوق رسالة في نافذة المكس (**Alt+K**) ستقرأ الإطارات من أعلى للأسفل **user32.dll!MessageBoxW** ← **coreclr.dll** ← **clrjit.dll** ← ... ← **MizanPro.dll** هنا تشرح للطلاب كيف يبدو **.NET** من الأسفل لا أسماء دوال C# ، ولا رموز، ومكتبة واحدة للنواة وواحدة للمترجم
- 4 في لوحة المعطيات، انظر وسائط **lpCaption**, **lpText**, **hWnd**, **uType**, **MessageBoxW** :أكثر اجعل النقطة **Log-only** (**Log text** = **{p:arg1}** ، Break = Never ، **Command** = **gc**) ⇒ نص الرسائل يصب في Log بلا توقف — « تحليل سلوكي » حي في 20 ثانية
- 5 ⇒ **Ctrl+F9** (Execute till return) تعود إلى ما بعد النداء، والقيمة في **EAX** (**IDOK** = 1) ، **IDCANCEL** = 2 ، **IDYES** = 6
- 6 **EAX** → **Modify** (أوفي شريط الأوامر (**set eax, 0**) ثم **F9** وانقر «OK» في الصندوق

النتيجة التطبيق يتصرف كما لو لم يتغير شيء: اسأل « ليش؟ » لأن `App.xaml.cs` يستدعي `MessageBox.Show(...)` ثم `args.Handled = true` بدون أن ينظر إلى النتيجة القرار اتخذ قبل الصندوق، والصندوق مجرد عرض

الجملة التي يجب أن تقال حرفيا . افحص وتأكد

« تعديل قيمة الإرجاع في حالة `MessageBox` يغير رد المستخدم ، لا قرار البرنامج الكسر الحقيقي يغير القرار إما الحساب الذي ينتجه (73 %، أو مصدر الثقة) `Registry/Al` (`state.enc`). لهذا الفصل ٧ هو الكسر، وهذا الفصل هو التفسير .
ثم: أضف « في برنامج يستخدم نتيجة الصندوق ليقرر (`RegisterNow() if (msg == IDYES)` كان تعديل `EAX` كسرا فعليا — هذا هو الفرق بين أن تعرف الأوامر وأن تعرف أين القرار .»

٨. ٣ متغيرات مفيدة للعرض (إن بقي وقت

- `bp MizanPro.dll:TokenProcessor.ParseAndVerify` — لا تعمل `x64dbg` لا يعرف رموز .NET. المدارة لهذا نرتق IL (بحث(بايتات بدل نقطة توقف بالاسم — قلها، فهي إجابة سؤال « ليش ما حطيت breakpoint على الدالة؟ ».
- `bpm <address>` (نقطة توقف على كتابة / قراءة :ذاكرة مفيدة في native ، وخادعة هنا لأن عناوين المتغيرات المدارة تتحرك مع ال GC.
- لو أردت « كسرا » بهذا الأسلوب في ميزان Pro: فعلا رتق `MessageBox.Show` إلى لا شيء (NOP) عند أول (بايت = إزالة ال nag فقط — وهو بالضبط اسمها في: الأدبيات `nag screen removal`.
- خيار « تجاهل الاستثناءات » عند `0xE0434352` يغير كم التوقيفات كثيرا أثناء هذه الخطوات — لا تنس ضبطه من الفصل ٣. ٣.

٩. الطريقة HxD — A على MizanPro.dll (الأكثر موثوقية، وأسهل (شرحاً

- 1 انسخ `C:\Lab\MizanPro\MizanPro.dll` إلى `backup\MizanPro.dll.orig` ، واعمل على نسخة: `copy MizanPro.dll MizanPro_cracked.dll`.
- 2 افتح النسخة في ← HxD ← `Edit → Search (Ctrl+F)` ← اختر ← **Hex values** اكتب `1F 49 D0` ← OK.
- 3 ستجد موزعا (أو أكثر داخل القسم `.text` غير البت :الأوسط `49` ← `01`.
- 4 احفظ (Ctrl+S). تأكد أن **الحجم لم يتغير** وهذا سبب نجاح :الطريقة لا جديد في جدول ال metadata ، ولا إعادة حساب لأطول أو — RVA فقط محتوى جسم .طريقة
- 5 ضع الملف المعدل مكان الأصلي (أوفي مجلد `cracked\` وشغل `MizanPro.exe` بدون المنقح.
- 6 نظف الحالة أولا (`reset-license.bat` ثم أدخل المفتاح المثبت `5N0FK-JQB1Z-7HSHF-N0P6D` ⇒ **تم .التفعيل** الآن الأثر :دائم أعد التشغيل مرتين، احذف ما شئت من `HKCU` — سيكتب التطبيق حالته من .جديد
- 7 أعد الملف الأصلي من `backup\` بعد العرض (وانسخه أيضا إلى مجلد .(التسليم

نقطتان سيصنعان فرقا في مناقشتك . تنبيه

(الا يوجد **Strong Name** ولا توقيع في مشروعنا — لو كان موجودا لرفض CLR تحميل الحزمة بعد التعديل، ولسقط التطبيق عند الإقلاع: قل « المشروع الحالي غير موقع؛ إضافة توقيع + فحص تكامل هي أول توصية في الفصل ١٢ » ، وأنت بذلك تجيب قبل أن يسأل (`DebugType=none` يعني لا — PDB لهذا لا يستطيع x32dbg أن يعرض لك أسماء المنفذات. الأصلية لا تعتذر عن: هذا قل إنه **مقصود** لتقريب الهدف من برنامج .حقيقي

٩. ٢ الطريقة dnSpyEx: B — أوضح تعليميا، وأدق تقنيا

- 1) File → Open → C:\Lab\MizanPro\MizanPro.dll ← exe وليس
- 2) الشجرة : MizanPro → MizanPro.Core.Engine → TokenProcessor → ParseAndVerify(string)
- 3) C# tab : للطلاب «واو» لحظة – الشرح تعليقات فيه بما كاملا الكود ستري
- 4) IL tab : بـ كله الطريقة جسم استبدل «Edit IL Instructions...» →
ldc.i4.1
ret
سيمر صحيحة وأبجدية 20 بطول مفتاح أي ← (17 2A : فقط بايتان)
- 5) File → Save Module → MizanPro_cracked.dll (من تأكد) «Don't use strong name» / Suppress PKT (ظهر إن)
- 6) وشغل dist\ في الملف بدل.

محرر C# في dnSpy يعيد ترجمة الكود بـ Roslyn ويحتاج كل المرجعيات (System.Runtime, WindowsBase...). في نشر self-contained بدون ملف واحد تكون كل المكتبات **بجانب** الـ DLL يعمل غالبا؛ لكن إن ظهر لك «could not load assembly» فاستخدم «Edit IL» كما فعلنا لا تقدم عرضا يعتمد على شيء جربته مرة واحدة فقط.

٩. ٣ الطريقة — C « الكسرالنظيف keygen :» بلا تعديل

لسنا ملزمين بتغيير برنامجنا لنثبت نقطة؛ أحيانا أوضح درس هو أن **الحماية نفسها قابلة للاشتقاق** (الفصل.١٠ الفرق بين الثلاثة يشرح دائما بنفس الجملة

الأسلوب	يغير ماذا	يدوم؟ التكامل؟	فحص يكتشفه	التعليمية قيمته
memory IL patch	في DLL صورة الذاكرة	(لا للجلسة)	يقيس شيء لا	ترجم متى + JIT يفسر الطريقة
file patch (HxD/dnSpy)	الحزمة ملف	نعم	نعم	نفحص / نوقع لماذا يفسر
keygen	— شيء لا الشرط نرضي	نعم	ما يوجد لا يفحص	الشرطان لماذا يفسر قاتلان الضعيفان

٩. ٤ كيف تتأكد أن ما تراه « كسر » وليس « صدفه »

- جرب مفتاحا **مرفوضا في الأصل** (مثل 5NOFK-... أو AAAAAA-BBBBBB-CCCCC-DDDDDD) — إن قبل بعد الرتق فهذا دليل سببي، لا مجرد « اشتغل ».
- جرب مفتاحا بطول خاطئ (12345 — يجب أن **يرفض** بعد كل الرتقات) لأن فحص الطول (يسبق ؛ هذا يظهر أنك لم تعطل الدالة كلها بل شرطا واحدا
- بعد الطريقة A/B: احذف ActivationBlob و state.enc ثم أعد التشغيل ⇒ يجب أن يعود إلى « تجربة » (الحالة لا تخزن في (الحزمة — يثبت أن الكسر يغير **القرار** لا **التخزين**
- وثق :بلقطة نافذة Ctrl+P في x32dbg تظهر الرتق المؤقت، وصورة الـ IL قبل / بعد من

١٠. ١ ما الذي تحسبه الدالة فعلا؟

بعد حذف الشروط والتحويل لأحرف كبيرة، المفتاح 20 محرفا من **A-Z0-9**: الشرطان **مجموع الأكواد** يقبل القسمة على 73 ، و **XOR التراكمي** يساوي **0x0A** لا ترتيب ولا موضع — أي تبديل للمحارف يبقي المفتاح. صالحا هذه معلومة: ثمينة تعني أن المساحة « مفطرة » بشرطين فقط، وأن أي تركيب يحققهما مقبول

برهان استحالة 0x2A — أربع سطور تنهي النقاش

الأرقام محارف 0-9 : $0x30..0x39 \rightarrow 6 \text{ البت} = 0$ ، $5 \text{ البت} = 1$
 الأحرف محارف A-Z : $0x41..0x5A \rightarrow 6 \text{ البت} = 1$ ، $5 \text{ البت} = 0$

محرفا 20 ل النهائي XOR في:

الأحرف عدد زوجية = 6 البت

الأرقام عدد زوجية = 5 البت

دائما 5 البت = 6 البت $\Rightarrow$ متساويتان الزوجيتان $\Rightarrow$ زوجي (زوجي) = 20 = الأرقام عدد + الأحرف وعدد

11 أو 00 : للبتين الممكنتان الوحيدتان القيمتان $\Rightarrow$

يمكن $\checkmark$ 5=0 البت ، 6=0 البت $\rightarrow 0x0A = 0b0000_1010$ المطلوب

(تناقض) مستحيل $\times$ 5=1 البت ، 6=0 البت $\rightarrow 0x2A = 0b0010_1010$ طلبت الأصلية المواصفة

ثغرة في « الحل » الذي اعتمد — اكتشفها الطلاب معنا • افحص وتأكد

من استحالة 0x2A نستنتج أن بتي ال XOR مرتبطان بأي أبجدية **A-Z0-9** ذات طول زوجي $\Rightarrow$ الحماية « لا تملك » سوى نصف فضاء البتات: والأخطر لأن $\text{bit0}(\text{xor}) = 0$ (القيمة 0x0A) فإن عدد المحارف الفردية زوجي $\Rightarrow$ مجموع الأكواد زوجي $\Rightarrow$ وبما أن المجموع يجب أن يقبل القسمة على 73 (فردي ، فاللازم المجموع يجب أن يكون مضاعفا ل 146 ، لا ل 73 القيم: الممكنة 1022 ، 1168 ، 1314 ، 1460 ، 1606 ، 1752) وكل مفتاح معروف عندنا مجموعه 1314 أو (1460 النتيجة: العملية كثافة المفاتيح الصالحة $\approx 1/4672 \approx 1/128 \times 1/146$ من فضاء 20 محرفا، لا 1/73. قل هذه على المسرح؛ إنها بالضبط نوع الملاحظة التي تظهر هندسة لا. ترحميا

١٠. ٢ الكود :الأول محاكاة الفحص) نفحص به أي (مفتاح

```

import string, random
ALPHA = string.ascii_uppercase + string.digits      # [A-Z0-9] (36 خيارا)
MOD, XOR_TARGET, LENGTH = 73, 0x0A, 20

def clean(key):
    return key.replace("-", "").upper()
def verify(key: str) -> bool:
    c = clean(key)
    if len(c) != LENGTH:
        return False
    if any(ch not in ALPHA for ch in c):
        return False
    total, x = sum(map(ord, c)), 0
    for ch in c:
        x ^= ord(ch)
    return total % MOD == 0 and x == XOR_TARGET

def fmt(key):
    return "-".join(key[i:i+5] for i in range(0, 20, 5))

if __name__ == "__main__":
    for k in ["M1Z4N-2025P-ROM4S-T3R01", "5N0FK-JQB1Z-7HSHF-NOP6D"]:
        c = clean(k); t = sum(map(ord, c))
        print(f"{k:24} sum={t:5d} %73={t%73:2d} valid={verify(k)}")

```

المخرجات المتوقعة: عندك السطر الأول valid=True ، والثاني valid=False — إن خرج العكس فأنت تكتب سكريبتا مختلفا.

١٠. ٣ الكسر: النظيف مولد يعمل في أقل من ثانية

— keygen.py (2/2) # القوة الغاشمة + نسخة « مفتاح مزخرف » باسم الفريق

```

import random

def brute_key():
    while True:
        k = "".join(random.choice(ALPHA) for _ in range(20))
        if verify(k):
            return k
            # ألف 13.8 قسناه ما أقصى محاولة، ألف 4.7~ متوسط

def vanity_key(prefix: str, tail: int = 5):
    '''ثوان-ميلي في يعمل - عشوائي ذيل عن وابحث (محرفا 15 حتى) بادئة ثبت'''
    p = "".join(ch for ch in prefix.upper() if ch in ALPHA)[:LENGTH - tail]
    p = p.ljust(LENGTH - tail, "X")
    while True:
        k = p + "".join(random.choice(ALPHA) for _ in range(tail))
        if verify(k):
            return fmt(k)

print(vanity_key("MIZAN2026X32DBG")) # → MIZAN-2026X-32DBG-B36SQ (مثبت)
print(brute_key())

```

مفاتيح جاهزة مولدة ومنتحقق منها (انسخها في العرض): **MIZAN-2026X-32DBG-B36SQ** · **GROUP-4SENI-OUR00-JYIP2** · **MIZAN-PROCR-ACKED-P4P7L** · **CRACK-MEW0R-KSHOP-BY0A5**
 — كلها $\text{sum} \in \{1314, 1460\}$ و $\text{xor} = 0x0A$ **ALBLA-SATEA-M4000-6LFK4**

١٠. ٤ تصحيح مهم في LabSheet.md (قله للمجموعة) المشرفة

المسودة: تقترح « ثبت 18 محرفا واشتق آخر 2 بحل المعادلتين . » هذا غالبا لا يحل القياس على 300 بادئة عشوائية بأبجدية [A-Z0-9] (مع فحص كل الاحتمالات للذيل، وليس عينة):

الخطة	البحث مساحة حل لها التي البادئات نسبة الحكم	
2 اشتق ← 18 ثبت	$36^2 = 1296$	7% ≈ × غير موثوق
3 اشتق ← 17 ثبت	$36^3 = 46656$	38% ≈ △ لا يكفي
4 اشتق ← 16 ثبت	$36^4 = 1.68M$	72% ≈ △ يبقى 28% بلا حل
ذيل ← 15 ثبت 5 عشوائي	محاولة ألف 4.7 ≈	100% (عمليا) ✓ استخدم هذا

لماذا تفشل « المعادلتان بجهولين » ؟ - ملاحظة

لأن القيدتين ليسا: مستقلين لكل محرف في [A-Z0-9] علاقة صارمة بين قيمته الحسابية وبتيه 5/6 ، فليست كل أزواج (مجموع، XOR) قابلة. للتحقيق الرياضيات النظيفة: تقول بمعلومية $a \oplus b = T$ فإن $a + b = T + 2(a \cdot b)$ $\Rightarrow$ المجموع محصور بين قيم قليلة لا تغطي كل المود 146.

هذا ليس « عيباً في الواجب - » هو اكتشاف تكتبه في: تقريرك « طبقنا طريقة الورقة ففشلناها في 93% من البادئات، وقسنا السبب وقترحنا البديل. « أقوى من حل التمرين. حرفياً

١٠. ٥ تمرين 5 دقائق للطلاب (أدره)هكذا

- 1 أعط كل طالب السطر الأول من `keygen.py` (20 سطرًا - لا تعطه. الحل
- 2: المطلوب مفتاح يبدأ باسمه هو (مثلاً) `SARAH2026...` بطول - 20 يستخدم `vanity_key` أو يعدل الذيل. يدويا
- 3 الأول الذي ينجح يشرح على السبورة كيف عرف أن المجموع يجب أن يكون زوجياً (قاعدة - (146 هذه هي. الدرجة
- 4 ثم اطلب فشلين: مقصودين مفتاح فيه - زائدة في الآخر، ومفتاح فيه حرف `s` صغير $\Rightarrow$ سيقبلهما التطبيق (+ ToUpperInvariant) إزالة (الشرطات - :اسأل « هل هذا آمن؟ ».
- 5 اختتم بأن كل من أنتج مفتاحاً في أقل من 30 ثانية يملك الآن إجابة سؤال « ما قيمة هذه الحماية؟ ».

11 ما قد يحرجك :غدا عيوب التطبيق التي رأيناها في اللقطات

ثلاثة أعطال من جذر واحد — وماذا تفعل إن لم تصلح قبل العرض

راجع `AUDIT_AND_FIXES.md` للتفصيل الكامل مع أرقام الأسطر هنا فقط ما يخص :العرض ما سيحدث على المسرح إن تركت الأخطاء كما هي

II. الجذر :الواحد أحداث تطلق أثناء `InitializeComponent`

أي عنصر XAML يعطي قيمة ابتدائية ويرتبط بمعالج حدث (`IsChecked` / `IsSelected` / `Text` + `Checked` / `SelectionChanged` / `TextChanged`) قد يعمل قبل أن تولد بقية عناصر الصفحة ⇒ `NullReferenceException` داخل منشئ الصفحة.

المستودع في (كما) الموضع	يحدث ماذا	الجمهور يراه ما
<code>Pages/CustomersPage.xaml:23</code> — <code>CardsToggle IsChecked="True"</code> <code>Checked="CardsToggle_Checked"</code>	« نافذة » ⇒ ينفجر المنشئ التي « متوقع غير خطأحدث <code>215233.png</code> لقطة في	خطأ :ممكن مشهد أسوأ نقرة أول في برمجي
<code>Pages/InvoicesPage.xaml:33-34</code> — <code>StatusCombo IsSelected="True"</code> <code>SelectionChanged="Filter_Changed"</code>	وجود قبل يعمل <code>LoadAsync</code> في يعاد الاستثناء ؛ <code>Loader</code> و يبقى ، <code>finally</code> ⇒ <code>_loading = true</code> للأبد يتجاهل لاحق <code>Refresh()</code> كل بصمت	« + فاضي فواتير جدول (3لقطة) «0 من 0-عرض
<code>Pages/ProductsPage.xaml:36-37</code> — على النمط نفس <code>CategoryCombo</code>	لا بطاقات، لا ⇒ العطل نفس والفلاتر ، « بيانات توجدلا » فارغة	لقطة (ميتة منتجات صفحة 4)
<code>Windows/NewInvoiceDialog.xaml:125-127</code> — <code>DiscountInput Text="0" TextChanged=...</code> → <code>UpdateSummary()</code> يلمس <code>TotalText</code> (سطر 134)	= « جديدةفاتورة » فتح آخر انهيار	أمام الزر هذا تضغط لا الذي الزر هو — الطلاب أن تريد « حين تلقائيا يفتح « فاتورة تريهم

قرار العرض (اختر)الآن . خطأ شائع – تجنبه

إما ينفذ ال Agent الإصلاح (حراس `if (!IsInitialized) return;` + ضبط القيم الابتدائية من الكود + `finally` (آمن قبل أن تصور اللقطات؛ أو تعد « مسارع عرض آمننا :» لوحة التحكم → الفواتير) ستبقى:فاضية لا تعد (!المعروض → التقارير → الإعدادات ولا تفتح العملاء / المنتجات / فاتورة .جديدة ملاحظة :دقيقة الإصلاح يغير لقطاتك .أيضا أعد التصوير بعد الإصلاح، بنفس الدقة، وإلا سيقارن أحدهم الصورة بالشاشة ويسألك عن الفرق

II. ٢ تراكب :النصوص `HorizontalAlignment="Left"` في واجهة RTL

في WPF مع `FlowDirection=RightToLeft` تعامل `Right` / `Left` `Start/End` معكوسة ، فكل زر « جديد » وكل صندوق شهر / سنة وكل قيمة في قائمة حالة قفزت فوق العنوان (لقطات 2 ، 4 ، 5). العلاج `Grid` بعمودين * و `Auto` بدل تراكب المحاذاة — تفصيله في `AUDIT_AND_FIXES.md` § ب 2./

II. ٣ أشياء « ليست أعطالا » لكنها ستفهم غلطا

- « إجمالي الإيرادات = « فاتورة واحدة :فقط الحساب صحيح (ملخص الشهر (الحالي ، والتسمية هي .المشكلة قلها بصوت عال قبل أن يسألها :أحد « هذاملخص الشهر، والبيانات مبذورة حول تاريخ التشغيل .»
- « المصروفات. ر. س :» `DataSeeder` يولد فواتير مبيعات فقط، فلا مشتريات — عيب بذور لا عيب محرك؛ سيختفي إن أضفنا المشتريات (مهمة 5 في قائمة ال Agent).
- التاريخ « ٢٦ / ٠٩ / ٢٠٢٦ » في جدول :الفواتير البذور نسبية لليوم ⇒ اللقطات القديمة تصبح .مستقبله أعد .التصوير
- `Select a date` بالإنجليزية داخل واجهة عربية + أرقام عربية ٢٠٢٦/١٠/٣ مع معرفات لاتينية :`INV-2026-00012` ثقافة غير مضبوطة؛ القرار في `AUDIT_AND_FIXES.md` § ب 3./
- « ٦ من ٧ أيام » ليست :`٧` هي الرقم العربي ٧ في خط — Windows لا تقع في فخ تصحيحها على .السبورة
- علامة « تنشيط:»Windows في كل لقطة، وفي وجهك على .المسرح نشط الجهاز أو استخدم VM .نظيفة

العرض الذي يظهر كسرا فقط يقول « نحن نعرف كيف نهاجم ». العرض الذي **يصلح** يقول « نحن مهندسون ». الجدول أدناه مبني على الثغرات التي استغللناها نحن. بالذات

استغللناه ما	الإصلاح	ينجح لماذا	الكلفة
الترخيص مفتاح أي على يعمل يرتبطلا (جهاز (بشيء	ECDSA-P256 :رقمي توقيع على {key HWID expiry plan} البائع، عند خاص بمفتاح داخل العام بالمفتاح والتحقق البرنامج	صالح مفتاح توليد يمكن لا بالجهاز والربط الخاص؛ بدون التسييب يوقف	(متوسطة خادم /شهادة (توقيع
دالة في القرار (واضحة واحدة ParseAndVerify()	أعد — bool ترجع لا للوظيفة تشفير فك مفتاح من يشق AES مفتاحمثلا (وافحص ، (الترخيص صحة فعلي تشفير بفك النجاح	لن «return true» الرتق الكسر مفيدا؛ شيئا يعيد لا شيء استخراج يحتاج قيمة إرجاع	لكن عالية فعالة
في مخزنة الحالة قابل بنص HKCU للتزوير	ProtectedData (DPAPI, CurrentUser) + من مشتق بمفتاح HMAC تسلسلي رقم + البصمة للتراجع مضاد	العملية خارج من التزوير ال المستخدم؛ مفتاح يحتاج إرجاع « يمنع counter » التاريخ	منخفضة
بمفتاح state.enc الملف داخل AES	الحزمة في يكون لا المفتاح: نفسه الترخيص من مشتق الخادم، عند منه التحقق بعد أو DPAPI	« مضمن بمفتاح تشفير » 6 في أثبتناه — أمني وهم خطوات	منخفضة
InstallDate وحيد كمرجع للتجربة	سجل في max(last_seen, now) + (Registry + ملف) مكانين GetSystemTimePreciseAsFileTime + monotonic ticks	الأيام استعادة يمكن لا فقط التاريخ بتغيير	منخفضة
DebugType=none IL تشويش، بلا حزمة نظيف، معروف بحجم	Obfuscation (Obfuscator/ConfuserEx) + NativeAOT أو لإزالة PublishReadyToRun IL	تبحث 1F 49 D0 يوجد لا آلة كود المهاجم على عنه؛ ترتيبه معاد	— متوسطة ٢. ١٢ انظر

منخفضة	فورا؛ يسقط الملف رتق يحتاج الذاكرة في الرتق بكثير أعلى جهدا	ملف SHA256 : الإقلاع عند لأمروليس (MizanPro.dll) موقعة؛ بقيمة ومقارنة (IL) تشغل لا: الاختلاف عند الدرجة الوظيفة	تكامل فحص لا إطلاقا ذاتي
+ منخفضة أحيانا كاذبة	التشغيل ويكشف الكلفة يرفع ك عليه تعتمد لا التلقائي؛ «حماية»	Debugger.IsAttached + CheckRemoteDebuggerPrese nt + قسير نداء توقيت + كقرار لا كإشارة — (rdtsc)	كشف يوجد لا منفح

١٢. أهم جملة دفاعية تقال في هذا المشروع

الحماية الحقيقية تخرج القرار من الجهاز لا يوجد طول مفتاح، ولا تشفير، ولا تشويش
يجعل الترخيص المحلي « مكسورا بصعوبة فقط » ؛ الخطة التي :تعمل ترخيص قصير العمر
من خادم (Token) بصلاحيه 24 (ساعة + صلاحية الوظائف تقرأ من الخادم + .تليمتري
عندها « كسر» التطبيق المحلي يعني :فقط نسخة تعمل يوما واحدا بلا .تحديث

١٢. ٢ « ميزان:»Pro v2 كيف نجعل ورشتنا نفسها أصعب (اقتراح للمستقبل

ثلاث نقاط تغلق الثغرات الثلاث الأكبر

1) نقل القرار من bool إلى « مفتاح وظيفه »

```
// بدل if (TokenProcessor.ParseAndVerify(key)) Activate();  
byte[] feat = Crypto.DeriveFeatureKey(keyMaterial, machineFingerprint);
```

2) قفل IL بالترجمة المسبقة (يتطلب إعادة نشر)

```
dotnet publish -c Release -r win-x86 --self-contained true -p:PublishAot=true  
# لا MizanPro.dll ولا تيار #Text: ما تبقى كود آله فقط ← الفصل 7 كله يصعب
```

3) فحص تكامل عند أول نداء حساس (وليس عند الإقلاع فقط)

```
var h = Convert.ToHexString(SHA256.HashData(File.ReadAllBytes(assembly.Location)));  
if (h != SignedHash) { /* بتلقائية تغلق لا الدرجة، الوظائف عطل */ }
```

تحذير :عملي **PublishAot** يلغي **System.Management** (WMI) في .NET 6. أي أن بصمة الجهاز يجب أن تنقل
إلى P/Invoke أو **GetAdaptersInfo** ؛ لا تقترح Aot في التقرير قبل أن تعرف .هذا

وإن أردت أن تبقى ورشة قابلة للحل . تنبيه

لا تجمع كل ما سبق؛ اختر **طبقتين** فقط (توقيع+ مفتاح .(وظيفة الهدف التعليمي أن يرى
الطلاب « ماذاغير ولماذا صار أصعب » ، لا أن يقفوا أمام .جدار اقترح على المشرف نسخة
«v2» كتحدى للموسم القادم — هذا أفضل ما يمكنك الخروج به من .العرض

١٣. | التوزيع

د	المقطع	يتكلم من	الشاشة على	الحاسم الفعل
0-3	ولماذا (والهدف المشكلة « وليس crackme ») حقيقيابرنامجا	علي	العنوان شريحة مخطط + الطبقات	تبدأ لا — <code>README.md</code> اعرض بالأدوات
3-7	الحماية بنيت كيف أرقام + طبقات ثلاث	ماجد	<code>TokenProcessor</code> في <code>.cs</code> VS Code	بصوت $1314 = 73 \times 18$ اذكر عال
7-11	لماذا + IL تشرح يفسد <code>PublishSingleFile</code> شيء كل	المعتصم	<code>csproj + dist\</code> شجرة <code>dnSpy</code> +	<code>MizanPro.dll</code> (exe لا) افتح أمامهم
11-14	تزوير 1: كسر + <code>ActivationBlob</code> <code>state.enc</code>	ماجد	<code>cmd + python</code>	/ بعد / قبل :لقطات ثلاث القيمة حذف
14-24	في IL رتق 2: كسر <code>x32dbg</code> ب الذاكرة	(المعتصم) + (يحرك) (علي) (يشرح)	<code>x32dbg</code> — لوحات CPU/Memory/Patc hes	<code>F12</code> → <code>Ctrl+B</code> → <code>1F 49 D0</code> → <code>Ctrl+E</code> → <code>F9</code> → مفتاح الرتق
24-28	لماذا (التوقيت درس (الثانية؟ المرة رفضت + JIT/tiering	علي	الشاشة نفس	عمدا بالمقلوب التجربة أعد
28-31	<code>MessageBoxW</code> 3: كسر + call stack + <code>EAX</code>	ماجد	<code>x32dbg Log</code>	كسب — كسرا لا تشرحا سمه أمانة نقطة
31-34	+ (HxD) الدائم الرتق حي مفاتيح مولد	المعتصم	<code>HxD + python</code>	من طالب باسم مفتاحا ولد الحضور
34-37	«ثغرة» جدول :الدفاع إصلاح + <code>NativeAOT</code> + التوقيع	علي	الفصل جدول 12	مهندسين نصبلك ذلك « :ب اختتم « أفضل

37	كيف + الأسئلة	الثلاثة	الختم شريحة	حدك دقيقة 38 — تزد لا
-3	مجموعات على سيشرفون			
8	الطلاب			

١٣. ٢ البروفة المسائية (افعلها الليلة 70 — دقيقة)

1 ابن: الحزمة

`dotnet publish -c Release -r win-x86 --self-contained true -p:PublishSingleFile e=false -o dist` ← تحقق أن MizanPro.dll موجود

2 `reset-license.bat` → شغل → سجل دخول → (admin/Admin@123) لاحظ الالفة « تجربة ٦ أيام ».

3 (الرتق المؤقت) الفصل (7 كاملا، **بالمؤقت** 60-90. ثانية. سجله

4 (التجربة العكسية) المفتاح قبل ال — (patch سجلها؛ ستحتاجها غدا إن توترت. يداك

5 الرتق الدائم عبر HxD على نسخة، واختبر `5N0FK-JQB1Z-7HSHF-NOP6D` بعد `reset-license.bat`.

6 شغل `keygen.py` واكتب مفتاحا باسم كل طالب في الصف الأمامي (5 ثوان لكل. (مفتاح

7 أعد كل شيء: لنظافته `reset-license.bat` + استبدل `MizanPro.dll` الأصلي + **أخرج من مجلد x32dbg**

8 صدر فيديو البروفة (1080p) إلى `C:\Lab\backup\demo.mp4` — هذا هو. تأمينك

شجرة الطوارئ (احفظ الأرقام . خطأ شائع — تجنبه

- A — كل شيء: يعمل الفصل 7 ثم 9 ثم 10.
- B — النمط `1F 49 D0` لم يظهر في: الذاكرة انتقل فوراً لـ HxD (الفصل 9: وقل « سنري الرتق على القرص لأنه أوضح » ، وعد للتوقيت في. الشرح
- C — dnSpy/HxD غير متاحين أو AV حذف: الملفات اعرض **keygen فقط** (الفصل 10) مقطع الفيديو المسجل + جدول الدفاع — وتكون قد غطيت 90% من. الرسالة
- D — المنقح نفسه رفض التشغيل على جهاز: العرض استخدم جهازك المحمول + وصل HDMI ، أو اعرض المقاطع المسجلة وشرح حي على. الشرائح **لا تعتذر أكثر من 10 ثوان**

١٣. ٣ أنت تشرف على 3 مجموعات — نظم ذلك هكذا

- **قاعدة: الأدوات** مجموعة تستخدم dnSpy لا IL ومجموعة تستخدم x32dbg للذاكرة ومجموعة للسلوك. (Procmon/log BP) لن تتكرر العروض وستتعلم أنت. الثلاثة
- **نقطتا تحقق فقط في البداية** (قبل أي: (كسر « أين دالة القرار؟ » و « مالم الذي تثبته رسالتك النهائية؟ » لا تسمح بأداة قبل هذين. الجوابين
- الأخطاء الثلاثة المتوقعة: منهم (1) يفتحون exe أحادي الملف ⇒ لا IL ؛ (2) يفتشون في

MizanPro.exe عن 1F 49 D0 ⇒ لن يجدوا؛ (3) يحاولون bp Namespace.Class.Method

⇒ لا يعمل على المدار راجع معهم هذه الثلاثة خلال أول 10 دقائق

- **سلم تقدير مختصر** (إن طلب :منك 30% تحليل صحيح للدالة، 25% إثبات سببي (قبل/ بعد) ، 20% أداة مختارة بتبرير، 15% دفاع واقتراح إصلاح، 10% عرض ووضوح
- امنح كل مجموعة 4 دقائق إلقائية + دقيقة واحدة « لو كنا مكانك لكسرنا .»... أنت تعلم، لا تحبط.

ليش x32dbg وليس x64dbg ؟

لأن الحزمة win-x86 ⇒ العملية 32 بت، وال debugger يجب أن يطابق عمارة الهدف ولو نشرنا win-x64 كان x64dbg.exe (لو سئلت « ليش ما نشرنا 64 ؟ : » لأننا أردنا أن ندرس الفرق بين المعمارتين. (عمليا

ليش ما كسرت native مباشرة؟

لأن منطق .NET يعيش في IL ، وكود الآلة يولده JIT عند الاستخدام وقد يعيد توليده (tiering). الرتق عند مصدر الحقيقة (IL) أصدق، وأسرع، وأقل هشاشة

كيف عرفت أن 1F 49 D0 هو المكان الصحيح؟

من dnSpy: ldc.i4.s 73 + rem = 1F 49 D0. تحققت في HxD على نفس الملف، ثم تأكدت أن البت التالي 2C (brfalse في الذاكرة

ليش ما جعلت الدالة كلها ret 1 ؟

فعلا فعلناها dnSpy: (ldc.i4.1; ret) وهي أمتن لأنها لا تعتمد على ثوابت لكن في الذاكرة نفضل تغيير بايت واحد لا يتغير طول الجسم، ولا يتحرك أي شيء، ولا يحتاج إعادة بناء

هل هذا قانوني؟

نكسر برنامجا كتبناه لغرض تعليمي؛ لا توزيع، لا استخدام ضد الغير، ولا آثار على أنظمة حقيقية — والورشة كلها تحت إشراف أكاديمي القانوني AUP / للمادة يذكر في التقرير

ما الفرق بين keygen و patcher ؟

ال keygen يرضي الشرط (لا يغير الهدف (إطلاقا ؛ ال patcher يعطل الشرط الأول أصعب رياضيا والأنظف، والثاني أصعب تقنيا وأسهل رياضيا

ليش sum % 73 وماشية؟

لأنها « مبهمة » لا « آمنة » القسمة على عدد أولي قرب 64 تجعل الشرط لا يبدو تافها، لكنها تحتفظ ببنية خطية (مجموع XOR + يمكن حلها — وهذا بالضبط درس « الغموض ≠ الأمان ».

ليش 73 بالذات؟

لا سبب. أممي ملاحظة :هندسية القبول الفعلي هو sum ∈ {1022,1168,1314,1460,1606,1752} لأن XOR=0x0A يفرض مجموعا زوجيا 146 ⇒ وليس 73.

ليش ما كشفك Antivirus ؟

لا شيء فعلناه يكتب على القرص في الفصل 7 (رتق في الذاكرة. (فقط وفي الفصل 9 نحن من نسخنا .ملفنا برمجيات الكراك الحقيقية تكتشف لأن سلوكها (حشوحقن، تعديلات على ملفات (الغير

نمطي.

لو أضفنا Strong Name ؟

سيرفض تحميل الحزمة بعد رتق القرص (توقيع ≠ محتوى ، وسيسقط الفصل 9 ؛ أما الفصل 7 فلن يتأثر) نتعامل مع الصورة في (الذاكرة ولهذا نوصي بالتوقيع + فحص تكامل .معا

ليش رتق ال MessageBox ما يعتبر كسرا؟

لأن `App.xaml.cs` يتجاهل نتيجة الصندوق؛ القرار اتخذ قبل أن يظهر تعديل EAX يغير العرض لا القرار — وشرحت ذلك للطلاب .صراحة

ليش لا تستخدم bpdll المكتوب في ديلكم؟

لأن هذا الأمر غير موجود في — x64dbg صحناه إلى `bp user32.MessageBoxW` بعد System Breakpoint (أو «Break on DLL load» كان يجب أن ينقح الدليل قبل الطباعة، وقد نقحنه.

كيف نتأكد أن المفتاح الذي تولدته « حقيقي » وليس صدفة؟

بنفس :الدالة أعيد حساب `sum` و `xor` وأقارنهما بما يقرؤه الكود المصدري؛ ثم أجرب المفتاح فعليا في النسخة الأصلية قبل الرتق (يجب أن (يرفض وبعده (يجب أن (يقبل

ما الذي لم تستطيعوا عمله؟ (الصراحة)تنقذك

لم نجرب حماية خارجية حقيقية، ولا كشف التوقيع ب anti-tamper وقت التشغيل، ولا unpacking ل VMProtect/Themida. مقترحنا للفصل :القادم نسخة v2 بتوقيع + مفتاح وظيفة (الفصل.12)

التنفيذ

المفتاح	يفعله ما	المفتاح	يفعله ما
F9	متابعة / تشغيل	F12	(pause) مؤقت إيقاف
F7	(step in) داخل خطوة	F8	(step over) فوق خطوة
Ctrl+F9	الطريقة من العودة حتى تنفيذ	Alt+F9	« المستخدم كود » حتى تشغيل
F4	المحدد السطر حتى تشغيل	Ctrl+F2	الهدف تشغيل إعادة
Alt+F2	الهدف إغلاق	Alt+A	جارية بعملية الإلحاق

النقاط والبحث والتعديل

الفعّل	أمر / مفتاح	الفعّل	أمر / مفتاح
هنا توقف نقطة	F2 / <code>bp <addr></code>	النقاط كل مسح	<code>bc</code>
تصحيح على نقطة ذاكرة كتابة / معماري	<code>bph</code> / <code>bpm</code>	الإصابة عند ينفذ أمر	يمين كليك → Edit → Command
أو عنوان إلى ذهاب اسم	Ctrl+G	الذاكرة في بحث	(نص) Ctrl+F (بايتات) Ctrl+B
الثنائية الذاكرة تحرير	Ctrl+E	اسطر تجميع	Space
تسمية / تعليق	<code>;</code> <code>::</code>	مرجعية علامة	Ctrl+D
الرتقات عرض	Ctrl+P	تعليقاتنا تحميل / حفظ	<code>dbsave</code> / <code>dbload</code>

اللوحات (View)

الاختصار	اللوحة	ورشتنا في نفعلها
Alt+C	CPU / Disassembly	للتعليق ; لتعديل، Space
Alt+M	الذاكرة خريطة	<code>MizanPro.dll</code> على العثور بوابة
Alt+K	Call Stack	<code>coreclr</code> / JIT من النداء أن إثبات
Alt+B	Breakpoints	مسح واحدة، مرة سجل، شروط،

الجمهور أمام « ير لم /رأى » دليل	Log	Alt+L
كثيرة خيوطا يضيف CLR لماذا	Threads	Alt+T
الجلسة في رتقناه ما كل	Patches	Ctrl+P
مع تبقى ملاحظات dbsave	Notes	Alt+N

تذكير • ملاحظة

بعض هذه الاختصارات قد يتغير بين إصدارات — x64dbg جربها في البروفة وثبت الذي يعمل عندك؛ الدقة أهم من السرعة.

— CMD / x32dbg انسخ والصق

```

===== البناء ( مرة واحدة ) =====
dotnet publish -c Release -r win-x86 --self-contained true -p:PublishSingleFile=false -o dist
dir dist\MizanPro.dll           :: شيء كل أوقف وإلا - يوجد أن يجب ::

===== البيئة ( في نفس النافذة ) =====
set DOTNET_TieredCompilation=0
set DOTNET_ReadyToRun=0

===== حالات الترخيص =====
reg delete "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /f
reg delete "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /f
del /f /q "%APPDATA%\MizanPro\state.enc"
reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob /t REG_SZ /d M1Z4N-2025P-ROM4S-T3R01 /f
reg add "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v InstallDate /t REG_SZ /d 20250101 /f
reg query "HKCU\Software\MizanSolutions\Mizan\RuntimeState" /v ActivationBlob

x32dbg ===== داخل =====
bp user32.MessageBoxW           :: ثم : Log text = {p:arg1} → Break Never → Command = gc
bp user32.MessageBoxIndirectW   :: وقاية ( هنا من تمر الرسائل بعض )
bp advapi32.RegSetValueExW      :: «التطبيق؟ يكتب متى» إثبات ::
Ctrl+B → 1F 49 D0              :: MizanPro.dll في ابحث ::
Ctrl+E → 49 إلى 01              :: الرتق ::
set eax, 0                      :: (كسرا ليس) الإرجاع قيمة تمرين ::

===== الملفات =====
C:\Lab\keygen.py                :: الفصل 10
C:\Lab\forge_state.py          :: الفصل 6
C:\Lab\reset-license.bat       :: الفصل 3.4
C:\Lab\backup\                 :: MizanPro.dll.orig + MizanPro_patched.dll

```

الأسماء العربية في المسارات (... \سنة4\Desktop تسبب مشاكل لبعض الأدوات عند الاستخراج؛ استخدم C:\Lab\ كما هنا.

البند	القيمة	أين من
طول المفتاح	(تجميلية الشرطيات) محرفا 20	TokenProcessor.cs:67
الأبجدية	تهم لا الأحرف حالة ، [A-Z0-9]	TokenProcessor.cs:68-73
الشرط 1	$\Sigma \text{ ASCII \% } 73 == 0$	TokenProcessor.cs:89
الشرط 2	(IL في مضمن const byte XOR == 0x0A)	TokenProcessor.cs:89 و 94
قاعدة مستنتجة	(زوجية) 73 لا 146 مضاعف المجموع	10.1 برهان
مفاتيح صالحة	M1Z4N-2025P-R0M4S-T3R01 · AAAAABBBBBCCCCCADN89 · MIZAN-2026X-32DBG-B36SQ · CRACK-MEWOR-KSHOP-BY0A5 · MIZAN-PROCR-ACKED-P4P7L	عددي فحص (10.3)
مفتاح « الرتق فقط »	5N0FK-JQB1Z-7HS HF-NOP6D ($\Sigma=1426$ ، $1426\%73=39$ ، XOR=0x0A)	قبل يرفض بعده يقبل الرتق،
مفتاح مرفوض موثق	AAAAA-BBBBBB-CCCCC-DDDDD ($\Sigma=1330$ ، $\%73=16$ ، XOR=0x04)	في الفشل مثال المسودات
نمط الرتق	1F 49 D0 ← 1F 01 D0 (واحدبايت)	\$2.2 ، \$7.2
بدائل الرتق	2C xx ← 90 90 (المجموع فرع) · مفتاح أي ⇒ الفرعين كلا · محرفا 20	\$7.4
مسار الحالة	HKCU\Software\MizanSolutions\Mizan\RuntimeState (ActivationBlob InstallDate)	ProductStateEngine.cs:47-49
ملف الحالة	AppData\Local\MizanPro\state.enc = Base64(AES-128-CBC(MIZAN_VALID <بصمة>))	ProductStateEngine.cs:168-171
AES	key Mizan2024_K3y128 · IV MizanPro_IV_2024 · PKCS7	ProductStateEngine.cs:59-62
البصمة	SHA256(MachineName UserName MAC) [0:8] → XXXX-XXXX-XXXX-XXXX	TokenProcessor.cs:106-117

Data/DataSeeder.cs	admin / Admin@123 · accountant / Acc@2024	الدخول
ProductStateEngine.cs:127-148	فقط الإقلاع عند تقرأ TrialPeriodDays أيام، 7	فترة التجربة

الإنجليزي	العربي	ورشتنا في المعنى
crackme	برنامج كسر تعليمي	MizanPro هدفنا
keygen	مولد مفاتيح	الرياضيات نفس من مفتاحا يشتق
patch / patcher	رتق / مرتق	القرار يغير واحد بايت
IL (Intermediate Language)	لغة الوسيط	نرتقه وما dnSpy تكتبه ما
JIT	الترجمة اللحظية	يحكم التوقيت لماذا تفسر
tiered compilation	الترجمة متعددة المستويات	محسن tier1 ثم سريع tier0
ReadyToRun / NativeAOT	ترجمة مسبقة	موجود غير IL تجعل
apphost	قشرة التشغيل	DLL يشغل الذي MizanPro.exe
single-file bundle	حزمة أحادية الملف	عندنا dnSpy فشل سبب
memory map	خريطة الذاكرة	البحث باب — Alt+M
binary pattern	نمط ثنائي	1F 49 D0
breakpoint (software)	نقطة توقف برمجية	int3 / CC
conditional / log-only BP	نقطة مشروطة / سجلية	8 الفصل في الأنافة
execute till return	تنفيذ حتى العودة	EAX لالتقاط — Ctrl+F9
call stack	مكدس النداءات	« نداء؟ من » إثبات — Alt+K
HWID / machine fingerprint	بصمة الجهاز	بالحهاز state.enc تربط
activation blob	كتلة التفعيل	Registry في القيمة اسم
trial period	فترة التجربة	أيام 7
nag screen	شاشة الإلحاح	« التجربة انتهت » صندوق

MizanPro يملكه لا ما	فحص التكامل integrity check
القرص رتق سمح غيابه	توقيع الحزمة strong name / signature
منع لا كلفة، رفع	تشويش / تمويه obfuscation
الأول الرخيص الإصلاح	تشفير بمفتاح المستخدم DPAPI / ProtectedData
الحالة تزوير يمنع	رمز مصادقة بمفتاح HMAC
تجاهله يجب الذي 0xE0434352	استثناء الفرصة الأولى first-chance exception

ملفات التسليم (نسخة ورشة) كاملة

- [AUDIT_AND_FIXES.md](#) — التقرير التحليلي للصور والكود ومهام الـ Agent.
- [Crack_Guide_x32dbg.pdf](#) — هذا الدليل (يوزع على الطلاب قبل .الورشة
- [MizanPro_Crack_Workshop.pptx](#) — شرائح العرض (38) (دقيقة مع ملاحظات .المتحدث
- [LabSheet.md](#) (بعدتصحيح التلميح الرياضي 10\$ — ٤) ورقة عمل .الطلاب
- [Workshop_Pack.bat](#) المحدث — يجمع [dist\](#) كاملا + الأدوات + الـ PDF.
- [tools/keygen.py](#) ، [tools/forge_state.py](#) ، [tools/reset-license.bat](#) .
- [backup/demo.mp4](#) — البروفة المسجلة (تذكرفي التقرير كجزء « قابلية التكرار .)»
- تقرير قصير (4-6): (صفحات الهدف، المنهجية، النتائج، الدفاع ، والإخفاقات بصدق — هذا الجزء يقرأه المشرف .فعلا

مصادر تذكر باسمها (لاتختلق روابط

- توثيق **x64dbg** الرسمي: (x64dbg.com/docs) الاختصارات، الأوامر [bp/bc/bph/bpm/log/msg/find/dbsave](#) ، وخيارات — Events/Exceptions كل أمر في هذا الدليل طابقناه معه
- وثائق مايكروسوفت «Single file deployment» و «Tiered compilation» و «Runtime configuration environment variables» ([*DOTNET_*](#) — للأسباب في 2. § ٤.
- مستودعات الأدوات: **dnSpyEx** (فرع الصيانة النشط لمشروع dnSpy (الأصلي ، **HxD** ، **PE-bear** ، **Sysinternals Procmon** نزلها من مستودعاتها الرسمية، وحفظ نسخة في [tools\](#) .للورشة
- مراجع المفاهيم: العامة فصل «Software Cracking» في موسوعات الهندسة العكسية، وكتب (E. Salmony) «*Reversing*» و «*Practical Malware Analysis*» (لأسلوب التحليل السلوكي الذي سرناه في الفصل 8).
- في داخلي: المشروع [README.md](#) (تصميم(الحماية ، [CrackGuide.md](#) (سيناريوالمشرف — بعد (تصحيحاته ، [.github/workflows/build.yml](#) (ماتغطيه CI وما لا تغطيه 11\$ — ٤ من تقرير .(التدقيق

قبل أن تصعد · افحص وتأكد

المفاتيح الخمسة مكتوبة في جيبك؟ · `dist\MizanPro.dll` موجود؟ · البروفة المسجلة
جاهزة؟ AV · معطل مؤقتاً على جهاز العرض؟ · `reset-license.bat` يعمل؟ · لا تضغط «
فاتورة جديدة» ؟

بالتوفيق — وإن سقط شيء، فالحل في الفصل 13. ٢ « / شجرة الطوارئ. » لا يوجد عرض
ناجح بلا خطة. بديلة